



Documentação do código

Minha Jornada Bíblica

Guia completo, em linguagem simples, para entender as telas, os dados, a sincronização e os arquivos do projeto.

Versão documentada

Código da branch main, após a integração com o Supabase. Documento preparado em 22 de julho de 2026.

Tecnologia	Papel no projeto
React	Monta as telas e reage aos cliques da usuária.
Vite	Executa o projeto durante o desenvolvimento e gera a versão de produção.
Tailwind CSS	Controla cores, espaçamentos e aparência.
Supabase	Guarda o plano e o histórico na nuvem.
localStorage	Mantém uma cópia no próprio navegador.

Objetivo deste material

Permitir que alguém sem conhecimento avançado em programação consiga localizar cada parte do projeto, entender o caminho dos dados e realizar pequenas manutenções com segurança.

Sumário

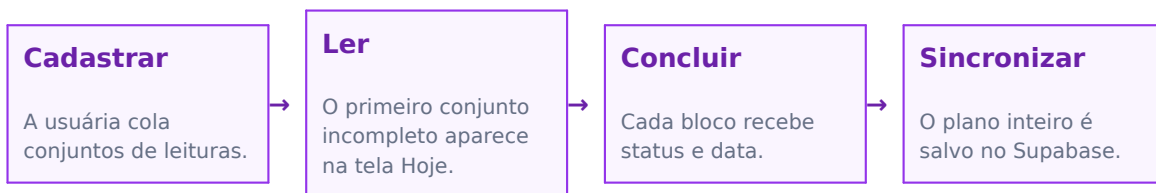
1. Visão geral do aplicativo
2. Estrutura de pastas e responsabilidade de cada arquivo
3. Como o aplicativo começa
4. Como os dados são organizados
5. Estados principais do componente App
6. Carregamento, migração e sincronização
7. Tela Hoje
8. Tela Calendário
9. Tela Meu Plano
10. Tela Progresso
11. Componentes auxiliares e navegação responsiva
12. Interface com o Supabase
13. Interface com o navegador e funcionamento offline
14. Aparência, ícone e instalação como aplicativo
15. Comandos de desenvolvimento e publicação
16. Cenários práticos
17. Pontos de atenção e melhorias futuras
18. Guia rápido para alterar o projeto
19. Glossário

Como ler

Não é necessário memorizar tudo. Comece pela visão geral e pelas telas. Use as demais seções como consulta quando precisar modificar ou corrigir alguma parte.

1. Visão geral do aplicativo

O aplicativo acompanha uma sequência contínua de leituras bíblicas. A sequência não fica presa a datas. A data só é registrada quando uma leitura é realmente concluída.



Regra central

A variável `plan` é a fonte principal da interface. Quando ela muda, o React redesenha as telas. Em seguida, o código grava a nova versão no navegador e no Supabase.

Ação	O que muda nos dados	Reflexo na tela
Marcar leitura	<code>isCompleted</code> vira <code>true</code> e <code>completedAt</code> recebe a data atual.	A leitura fica riscada e entra no histórico.
Desfazer	<code>isCompleted</code> vira <code>false</code> e <code>completedAt</code> volta a <code>null</code> .	A leitura retorna ao conjunto atual.
Editar	O campo <code>title</code> do bloco é substituído.	O novo texto aparece em todas as telas.
Apagar conjunto	O conjunto é removido do vetor <code>plan</code> .	Somem a sequência e o histórico ligado a ela.
Apagar histórico	Todos os blocos voltam a não concluídos.	A sequência permanece, mas o progresso zera.

Em uma frase

O aplicativo é uma tela React que lê e altera um grande objeto chamado `plan`; esse objeto é copiado para dois lugares: Supabase e `localStorage`.

2. Estrutura de pastas

```
minha-jornada-biblica/  
├─ index.html  
├─ package.json  
├─ package-lock.json  
├─ vite.config.js  
├─ tailwind.config.js  
├─ postcss.config.js  
├─ public/  
│   └─ manifest.webmanifest  
│       └─ icons/  
│           ├── favicon-32.png  
│           ├── icon-192.png  
│           └─ icon-512.png  
└─ src/  
    ├── main.jsx  
    ├── App.jsx  
    ├── index.css  
    └─ lib/  
        └─ supabase.js
```

Arquivo	Explicação simples
index.html	É a página-base. Contém o ponto onde o React será encaixado e as referências aos ícones.
package.json	Lista bibliotecas e comandos do projeto.
package-lock.json	Fixa versões exatas para instalações previsíveis.
vite.config.js	Ativa o suporte do Vite ao React.
tailwind.config.js	Informa ao Tailwind quais arquivos contêm classes de estilo.
postcss.config.js	Liga Tailwind e Autoprefixer ao processamento do CSS.
src/main.jsx	Ponto inicial do JavaScript. Coloca App dentro da página.
src/App.jsx	Arquivo principal: telas, regras, cliques e sincronização.
src/index.css	Importa Tailwind e define duas animações.
src/lib/supabase.js	Cria a conexão reutilizável com o Supabase.
manifest.webmanifest	Define nome, cores e ícones ao instalar o site no celular.

3. Como o aplicativo começa

3.1 index.html

O navegador abre primeiro o index.html. A linha abaixo cria um espaço vazio chamado root:

```
<div id="root"></div>
```

Depois, o navegador carrega src/main.jsx:

```
<script type="module" src="/src/main.jsx"></script>
```

3.2 main.jsx

O main.jsx procura o elemento root e coloca o componente App dentro dele.

```
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
)
```

O que é um componente?

É uma função que devolve uma parte visual da página. O componente App coordena tudo. Os componentes HojeView, CalendarioView, PlanoView e ProgressoView representam telas menores.

3.3 StrictMode

O StrictMode ajuda a encontrar problemas durante o desenvolvimento. Ele não aparece visualmente para a usuária. Em desenvolvimento, alguns efeitos podem ser executados mais de uma vez para detectar comportamentos inseguros.

4. Como os dados são organizados

O plano é um vetor de conjuntos. Cada conjunto possui um vetor de blocos. Cada bloco representa uma leitura.

```
[
  {
    id: "set-1",
    blocks: [
      {
        id: "b1",
        title: "Gênesis 1-3",
        isCompleted: false,
        completedAt: null
      }
    ]
  }
]
```

Campo	Tipo	Significado
plan	vetor	Todos os conjuntos, na ordem de leitura.
set.id	texto	Identificador único do conjunto.
set.blocks	vetor	Leituras que pertencem ao conjunto.
block.id	texto	Identificador único do bloco.
block.title	texto	Referência bíblica exibida.
block.isCompleted	booleano	true significa concluída; false significa pendente.
block.completedAt	data ou null	Momento da conclusão; null significa sem data.

Por que guardar a data dentro do bloco?

O histórico não é uma lista independente. Ele é calculado olhando os blocos concluídos e agrupando seus valores de completedAt por dia. Isso mantém o plano e o histórico coerentes.

Consequência importante

Ao desfazer uma leitura, completedAt volta a null e ela desaparece do histórico. Ao apagar um conjunto, também desaparecem todas as datas que estavam nos blocos daquele conjunto.

5. Estados principais do componente App

Estado é uma informação que o React acompanha. Quando o estado muda, a tela é atualizada automaticamente.

Estado	Valor inicial	Para que serve
activeTab	'hoje'	Controla qual tela está visível.
plan	[]	Guarda todos os conjuntos e blocos.
showCompletedSet	false	Mostra ou esconde a mensagem de conjunto concluído.
isLoading	true	Mantém a tela de carregamento enquanto os dados são buscados.
isInitialized	false	Impede salvar antes de concluir o carregamento inicial.
syncStatus	'loading'	Informa carregando, salvando, salvo ou sem sincronização.

currentSet

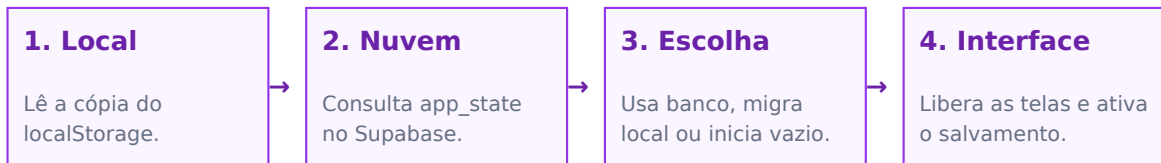
currentSet não é um estado separado. Ele é calculado a partir de plan. O código procura o primeiro conjunto que ainda tenha algum bloco não concluído.

```
const currentSetIndex = plan.findIndex(
  set => set.blocks.some(block => !block.isCompleted)
);
```

Isso implementa a regra principal: se um dia não houver leitura, o aplicativo não pula nada. O mesmo conjunto continua sendo o atual.

6. Carregamento, migração e sincronização

6.1 Ordem do carregamento



Situação	Decisão do código
Supabase contém um plano	O plano do Supabase prevalece e atualiza a cópia local.
Supabase vazio e navegador contém plano	O plano local é exibido e enviado ao Supabase.
Ambos vazios	O aplicativo começa sem sequências.
Falha ao consultar Supabase	A cópia local é usada e o status fica offline.

6.2 Por que isInitialized existe?

Sem essa trava, o primeiro valor de plan seria o vetor vazio. O efeito de salvamento poderia gravar esse vazio no banco antes de a consulta terminar. isInitialized só vira true ao final do carregamento.

6.3 Salvamento com espera de 500 ms

Quando plan muda, o aplicativo salva imediatamente no localStorage e espera 500 milissegundos antes de enviar ao Supabase. Se outra mudança acontecer nesse intervalo, a tentativa anterior é cancelada.

```
window.setTimeout(async () => {
  await supabase
    .from('app_state')
    .update({ plan, updated_at: new Date().toISOString() })
    .eq('id', 1);
}, 500);
```

6.4 Retorno da internet

O navegador dispara o evento online quando a conexão volta. O aplicativo então envia a versão atual de plan novamente.

Modelo de sincronização atual

É um modelo simples de última gravação vence. Como apenas uma pessoa usa o aplicativo, ele é suficiente no momento. Se dois dispositivos editarem ao mesmo tempo, a última gravação pode substituir a anterior.

7. Tela Hoje

A tela Hoje mostra apenas o primeiro conjunto ainda incompleto.

Elemento	Origem	Comportamento
Contador	Blocos concluídos / total	Exibe, por exemplo, 1 de 3 concluídas.
Barra de progresso	Percentual calculado	A largura cresce conforme os blocos são marcados.
Cartão de leitura	block.title	Mostra a referência bíblica.
Botão de check	toggleBlock	Conclui o bloco e registra a data.
Botão Desfazer	toggleBlock com undo=true	Limpa status e data.
Mensagem de conclusão	showCompletedSet	Aparece quando todos os blocos do conjunto ficam concluídos.

Função toggleBlock

A função percorre todos os conjuntos, encontra o conjunto correto e depois encontra o bloco correto. Ela cria novas cópias dos objetos em vez de alterar o conteúdo antigo diretamente. Essa é a maneira recomendada de atualizar estado no React.

```
b.id === blockId
? {
  ...b,
  isCompleted: !undo,
  completedAt: !undo ? now : null
}
: b
```

Por que usar ...b?

Os três pontos copiam todos os campos existentes do bloco. Depois, somente isCompleted e completedAt são substituídos.

8. Tela Calendário

Apesar do nome do componente ser `CalendarioView`, a interface atual funciona como uma linha do tempo do histórico, e não como uma grade mensal.

Como o histórico é construído

- Percorre todos os conjuntos.
- Percorre todos os blocos.
- Seleciona somente blocos concluídos que tenham `completedAt`.
- Usa a parte AAAA-MM-DD da data para formar grupos diários.
- Ordena do dia mais recente para o mais antigo.

```
if (block.isCompleted && block.completedAt) {  
  const dateStr = block.completedAt.split('T')[0];  
  dates[dateStr].push(block.title);  
}
```

useMemo

`useMemo` evita refazer o agrupamento em toda atualização visual. O histórico só é recalculado quando `plan` muda.

Apagar histórico

O botão pede confirmação e percorre todos os blocos, definindo `isCompleted` como `false` e `completedAt` como `null`. A sequência e os títulos são preservados.

Não existe uma tabela separada de histórico

A tela cria o histórico em tempo real a partir dos blocos. Portanto, zerar o histórico significa zerar as conclusões.

9. Tela Meu Plano

Essa tela reúne três funções: resumo geral, administração da sequência e importação de novos conjuntos.

9.1 Progresso geral

Conta todos os blocos e calcula a porcentagem de concluídos.

```
percentage =
  totalBlocks === 0
    ? 0
    : Math.round((completedBlocks / totalBlocks) * 100);
```

9.2 Editar conjunto

Ao clicar em Editar, os títulos do conjunto são copiados para `editingTitles`. Os campos de texto alteram essa cópia temporária. Salvar chama `handleEditSet`; Cancelar descarta a cópia.

Estado local	Função
<code>editingSetId</code>	Identifica qual conjunto está em modo de edição.
<code>editingTitles</code>	Guarda temporariamente os textos digitados.
<code>pasteText</code>	Guarda o conteúdo da caixa de importação.

9.3 Apagar conjunto

`handleDeleteSet` usa `filter` para criar um novo plan sem o conjunto escolhido. Há uma confirmação antes da remoção.

9.4 Importar plano

Cada linha digitada vira um conjunto. Dentro da linha, o caractere `|` separa os blocos.

```
Gênesis 1-3 | Salmos 1-2 | Mateus 1-2
Gênesis 4-6 | Salmos 3-4 | Mateus 3-4
```

Os identificadores usam `Date.now()`, que fornece o horário atual em milissegundos, combinado com a posição da linha e do bloco.

10. Tela Progresso

A tela Progresso calcula três indicadores. Nenhum deles fica salvo separadamente; todos são derivados de plan.

Indicador	Como é calculado
Dias ativos	Quantidade de datas diferentes encontradas em completedAt.
Leituras concluídas	Quantidade de blocos com isCompleted igual a true.
Conjuntos completos	Quantidade de conjuntos em que todos os blocos estão concluídos.

Por que usar Set para dias ativos?

Um Set não permite valores repetidos. Se três leituras forem concluídas no mesmo dia, a data entra uma única vez. Por isso, o tamanho do Set representa a quantidade de dias ativos.

```
const readDays = new Set();
readDays.add(block.completedAt.split('T')[0]);
const activeDays = readDays.size;
```

Vantagem de calcular em vez de armazenar

Os números sempre refletem o plano atual. Se uma leitura for desfeita ou um conjunto apagado, as estatísticas se corrigem automaticamente.

11. Componentes auxiliares e navegação

SyncStatus

Transforma syncStatus em texto, cor e ícone.

Valor	Mensagem	Significado
loading	Carregando...	O aplicativo ainda busca os dados iniciais.
saving	Salvando...	Existe uma atualização sendo enviada.
saved	Dados salvos	A última gravação foi aceita pelo Supabase.
offline	Sem sincronização...	O banco não respondeu; a cópia local foi preservada.

NavItem e MobileNavItem

São componentes pequenos para evitar repetir os mesmos estilos dos botões de navegação. Recebem quatro propriedades: active, icon, label e onClick.

Desktop x celular

Tela	Navegação
Desktop	Menu lateral fixo, visível a partir do tamanho md.
Celular	Barra fixa na parte inferior, escondida a partir do tamanho md.

As classes hidden, md:flex e md:hidden são responsáveis por essa troca. Isso é chamado de design responsivo.

12. Interface com o Supabase

12.1 Cliente Supabase

src/lib/supabase.js cria um objeto chamado supabase. Esse objeto concentra as chamadas ao banco.

```
import { createClient } from '@supabase/supabase-js';

export const supabase = createClient(
  VITE_SUPABASE_URL,
  VITE_SUPABASE_PUBLISHABLE_KEY
);
```

O código permite usar variáveis VITE_SUPABASE_URL e VITE_SUPABASE_PUBLISHABLE_KEY. Se elas não existirem no ambiente, usa os valores públicos configurados no arquivo. A chave administrativa não é usada.

12.2 Tabela app_state

Coluna	Tipo	Uso
id	integer	Sempre 1. Garante uma única linha para o aplicativo.
plan	jsonb	Guarda o vetor completo do plano.
updated_at	timestampz	Registra a última tentativa de atualização aceita.

12.3 Operações utilizadas

Operação	Código	Finalidade
SELECT	.select('plan').eq('id', 1).single()	Carrega o plano ao iniciar.
UPDATE	.update({ plan, updated_at }).eq('id', 1)	Substitui o plano salvo.

12.4 Políticas de acesso

Como ainda não há login, a função anônima possui permissão de leitura, inserção e atualização da linha 1. Isso permite que o site funcione diretamente no navegador.

Limitação de segurança

A publishable key é pública por definição. Sem login, qualquer pessoa que obtenha a configuração e entenda a API pode tentar acessar a linha permitida. Não guarde dados sensíveis. Quando o aplicativo crescer, adicione autenticação e políticas ligadas a auth.uid().

13. Interface com o navegador

13.1 localStorage

localStorage é um pequeno armazenamento do próprio navegador. A chave usada é:

```
minhaJornadaBiblica_plan
```

O conteúdo é convertido entre objeto JavaScript e texto JSON:

```
JSON.stringify(plan) // objeto para texto  
JSON.parse(savedPlan) // texto para objeto
```

13.2 Para que serve a cópia local?

- Mostrar o último plano conhecido quando o Supabase não responder.
- Migrar o histórico antigo para o banco na primeira abertura.
- Evitar perda imediata quando ocorrer falha de conexão.

13.3 O que significa offline na versão atual?

A interface continua acessível com os dados locais. Alterações também modificam a cópia local. Quando o evento online ocorre, o aplicativo tenta enviar o plan atual.

Cuidado com vários dispositivos

Se um celular ficar offline enquanto outro aparelho altera o banco, ao reconectar o celular antigo pode enviar sua cópia e substituir dados mais novos. Uma fila de alterações ou controle de versão seria uma melhoria futura.

14. Aparência, ícone e instalação

Tailwind CSS

A maioria dos estilos está escrita diretamente no atributo `className` dos elementos JSX. Exemplos:

<code>bg-[#faf5ff]</code>	fundo lilás muito claro
<code>text-purple-800</code>	texto roxo escuro
<code>rounded-3xl</code>	cantos bem arredondados
<code>shadow-sm</code>	sombra leve
<code>md:hidden</code>	esconde em telas médias ou maiores

index.css

Importa as camadas do Tailwind e cria duas animações: `fadeIn`, que faz o conteúdo aparecer suavemente, e `bounceSubtle`, que move levemente a mensagem de conclusão.

Ícones

Os ícones internos dos botões vêm da biblioteca `lucide-react`. Já o ícone do aplicativo está em três tamanhos na pasta `public/icons`.

manifest.webmanifest

Permite que navegadores compatíveis apresentem o site como aplicativo instalável. Define nome, cor de fundo, cor do tema, modo standalone e imagens de 192 e 512 pixels.

Arquivo	Uso
favicon-32.png	Ícone pequeno da aba do navegador.
icon-192.png	Atalho e ícone para celular.
icon-512.png	Ícone de alta resolução para instalação.

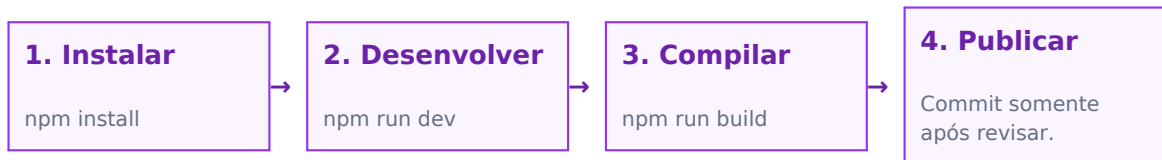
15. Comandos de desenvolvimento

Comando	O que faz
npm install	Baixa as bibliotecas listadas no package.json.
npm run dev	Abre o servidor de desenvolvimento do Vite.
npm run build	Gera a versão otimizada dentro da pasta dist.
npm run preview	Abre localmente a versão já construída.

Dependências principais

Pacote	Função
react / react-dom	Componentes, estados e renderização da interface.
@supabase/supabase-js	Comunicação com o Supabase.
lucide-react	Ícones dos botões e do status.
vite	Servidor e processo de build.
tailwindcss	Classes utilitárias de estilo.
postcss / autoprefixer	Processamento e compatibilidade do CSS.

Sequência segura para testar mudanças



16. Cenários práticos

Primeira abertura com banco vazio e navegador com dados

O aplicativo lê o localStorage, encontra o Supabase vazio, mostra o plano local e envia esse plano ao banco.

Primeira abertura em outro aparelho

O localStorage está vazio, mas o Supabase contém plan. O aplicativo baixa o plano e cria uma nova cópia local.

Marcar uma leitura

toggleBlock altera o bloco. React redesenha a tela. O efeito salva localmente e, após 500 ms, atualiza o Supabase.

Falha na internet durante uma alteração

A cópia local recebe a mudança. A gravação remota falha e o status offline aparece.

Internet volta

O evento online chama uma nova atualização com o plan atual.

Apagar o último conjunto

plan vira um vetor vazio. O novo código salva inclusive o vetor vazio no localStorage e no Supabase.

Editar um título já concluído

O status e a data permanecem. O histórico passa a mostrar o novo título.

17. Pontos de atenção e melhorias futuras

Tema	Situação atual	Melhoria possível
Segurança	Sem login; acesso anônimo à linha 1.	Adicionar autenticação e RLS por usuário.
Conflitos	Última gravação vence.	Comparar updated_at ou usar número de versão.
Offline	Cópia local e nova tentativa ao reconectar.	Criar fila de mudanças pendentes.
Banco	Plano inteiro em um único JSON.	Separar planos, conjuntos, blocos e histórico em tabelas se crescer.
Calendário	Linha do tempo por data.	Criar uma grade mensal real.
Testes	Verificação manual e build.	Adicionar testes automatizados para funções e telas.
Mensagens	alert e confirm do navegador.	Criar caixas de diálogo próprias e mais bonitas.

A arquitetura atual é adequada ao tamanho do projeto

Guardar tudo em um único JSON simplifica muito a integração e a manutenção. A separação em várias tabelas só se torna necessária quando houver múltiplos usuários, consultas mais avançadas ou grande volume de dados.

18. Guia rápido para alterar o projeto

Mudar textos ou cores

- Procure o texto ou a classe Tailwind em `src/App.jsx`.
- Para cores gerais do navegador e instalação, confira `index.html` e `manifest.webmanifest`.

Criar uma nova tela

- Crie um componente semelhante a `HojeView`.
- Adicione um novo valor para `activeTab`.
- Adicione o botão no menu desktop e no menu móvel.
- Renderize o componente conforme `activeTab`.

Adicionar um novo campo ao bloco

- Inclua o campo quando `handlePastePlan` cria os blocos.
- Considere dados antigos, que ainda não terão esse campo.
- Atualize as funções de edição, exibição e estatística que precisarem dele.

Alterar a integração com o banco

- A conexão fica em `src/lib/supabase.js`.
- O carregamento está no primeiro `useEffect` de `App`.
- O salvamento está no segundo `useEffect` de `App`.
- Nunca coloque a `service_role` no navegador.

Checklist antes de publicar

- Executar `npm install` se as dependências mudaram.
- Executar `npm run build`.
- Testar importação, conclusão, edição, exclusão e histórico.
- Confirmar que o status chega a `Dados salvos`.
- Conferir a coluna `plan` no Supabase.
- Revisar o diff e confirmar o commit.

19. Glossário

Termo	Significado simples
API	Forma padronizada de um sistema conversar com outro.
Array / vetor	Lista ordenada de valores. plan e blocks são vetores.
Booleano	Valor que só pode ser true ou false.
Build	Processo que cria a versão otimizada para publicação.
Componente	Função React que representa uma parte visual.
Estado	Valor acompanhado pelo React; ao mudar, atualiza a interface.
Hook	Função especial do React, como useState, useEffect e useMemo.
JSON	Formato de texto usado para representar objetos e listas.
JSX	Sintaxe que permite escrever estruturas parecidas com HTML no JavaScript.
localStorage	Armazenamento simples dentro do navegador.
PWA	Site com recursos de aplicativo instalável.
Publishable key	Chave pública usada pelo cliente, limitada pelas políticas do banco.
RLS	Regras do PostgreSQL que controlam quais linhas podem ser acessadas.
Supabase	Serviço de banco PostgreSQL, API e autenticação.
Vite	Ferramenta que executa e compila o projeto.

Resumo final

Se você lembrar apenas de uma ideia, lembre desta: plan contém tudo. As telas leem esse valor; os botões o alteram; os efeitos o copiam para o navegador e para o Supabase.